MACHINE LEARNING

LAB EXPERIMENT / OBSERVATION

Name	Iqbal Ahmed
Register Number	192421320
Course Code	ITA0614
Course Name	Machine Learning

Experiment Question

Implement and demonstrate the FIND-S algorithm for finding the most specific hypothesis based on a given set of training data samples.

This observation contains the aim, procedure, algorithm, five worked example problems, Python code, sample input, sample output, and result.

1. Aim

To implement and demonstrate the FIND-S algorithm in Python to find the most specific hypothesis that is consistent with all positive training examples in a given dataset.

2. Basic Concept

FIND-S is a concept-learning algorithm. It starts with the most specific possible hypothesis and generalises it only when a positive training example requires a change. Attributes that match remain specific; attributes that differ are replaced by **?**, meaning that any value is accepted for that attribute. Negative examples are ignored by the basic FIND-S algorithm.

Symbols used

- **Ø** — the most specific initial value, before any positive example is processed.
- **?** — any value is acceptable for that attribute.
- **Positive example** — a training sample labelled Yes.
- **Negative example** — a training sample labelled No; FIND-S does not use it to update the hypothesis.

3. Procedure

1. Create a training dataset with attribute values and a target class (Yes/No).
2. Initialise the hypothesis to the most specific state, represented by Ø for every attribute.
3. Read the training examples one by one.
4. If an example is negative, leave the hypothesis unchanged.
5. If an example is positive and the hypothesis is still Ø, copy the example's attribute values into the hypothesis.
6. For each later positive example, compare each attribute with the current hypothesis: if the values match, keep the value; if they differ, replace it with ?.
7. Continue until all examples have been processed.
8. Display the final hypothesis.

4. Algorithm / Pseudocode

```
FIND-S(training_data):
    hypothesis = [Ø, Ø, ..., Ø]

    for each example in training_data:
        if example.target == "Yes":
            if every attribute in hypothesis is Ø:
                hypothesis = example.attributes
            else:
                for each attribute i:
                    if hypothesis[i] != example[i]:
                        hypothesis[i] = "?"

    return hypothesis
```

Note: The initial Ø check is implemented in the Python code using a Boolean flag so that a real attribute value is not confused with the initial hypothesis.

5. Five Example Problems

Example 1 — EnjoySport dataset (step-by-step)

Problem: Find the most specific hypothesis for the following training samples. The attributes are Sky, AirTemp, Humidity, Wind, Water, and Forecast.

Sky	AirTemp	Humidity	Wind	Water	Forecast	EnjoySport
Sunny	Warm	Normal	Strong	Warm	Same	Yes
Sunny	Warm	High	Strong	Warm	Same	Yes
Rainy	Cold	High	Strong	Warm	Change	No
Sunny	Warm	High	Strong	Cool	Change	Yes

Hypothesis update

Step	Training sample	Hypothesis after processing	Reason
Initial	—	■Ø, Ø, Ø, Ø, Ø, Ø■	Start with the most specific hypothesis.
1	Example 1: Yes	■Sunny, Warm, Normal, Strong, Warm, Same■	First positive sample is copied.
2	Example 2: Yes	■Sunny, Warm, ?, Strong, Warm, Same■	Humidity differs, so replace it with ?.
3	Example 3: No	Unchanged	Negative samples are ignored.
4	Example 4: Yes	■Sunny, Warm, ?, Strong, ?, ?■	Water and Forecast differ from earlier positive samples.

Final hypothesis: ■Sunny, Warm, ?, Strong, ?, ?■

Interpretation: the positive examples all have Sunny sky, Warm air temperature, and Strong wind. The positive examples have different humidity, water, and forecast values, so those attributes become ?. The negative example does not change the hypothesis.

Example 2 — Student passes an exam

Attributes: Attendance, StudyHours, Assignment. Find the hypothesis from the following samples.

Attendance	StudyHours	Assignment	Pass
High	Many	Submitted	Yes
High	Few	Submitted	Yes
Low	Many	Submitted	No
High	Many	Missing	Yes

Final hypothesis: ■High, ?, ?■

Explanation: The two positive examples disagree on StudyHours, and the later positive example differs on Assignment. Attendance remains High. The negative sample is ignored.

Example 3 — Outdoor activity

Attributes: Weather, Temperature, Wind. Find the hypothesis for the activity labelled Yes/No.

Weather	Temperature	Wind	Activity
Clear	Warm	Low	Yes
Clear	Warm	High	Yes
Cloudy	Cold	Low	No
Clear	Cool	Low	Yes

Final hypothesis: ■Clear, ?, ?■

Explanation: Weather stays Clear across all positive samples. Temperature and Wind vary, so both become ?. The negative sample is ignored.

Example 4 — Loan pre-screening demonstration

Attributes: Income, CreditHistory, Employment. Find the hypothesis for a simplified Approved label. This is only an algorithm example, not a real lending rule.

Income	CreditHistory	Employment	Approved
High	Good	Stable	Yes
High	Good	Temporary	Yes
Low	Poor	Stable	No
Medium	Good	Stable	Yes

Final hypothesis: ■?, Good, ?■

Explanation: Income varies among positive samples, so it becomes ?. CreditHistory remains Good. Employment varies, so it becomes ?. The negative sample is ignored.

Example 5 — Water sample alert

Attributes: Turbidity, pHRange, Conductivity. Find the hypothesis for a sample labelled Alert/NoAlert.

Turbidity	pHRange	Conductivity	Class
High	Normal	High	Alert

Turbidity	pHRange	Conductivity	Class
High	Normal	Low	Alert
Low	Acidic	High	NoAlert
Medium	Normal	High	Alert

Final hypothesis: ■?, Normal, ?■

Explanation: Turbidity and Conductivity vary across positive examples, so they become ?. pHRange remains Normal. This toy dataset demonstrates FIND-S only and is not a water-safety classifier.

6. Python Program

The program below reads a dataset as a list of examples. Each example contains attribute values followed by its target label. Only examples labelled Yes are used to update the hypothesis.

```
def find_s(training_data, positive_label="Yes"):
    """Return the most specific hypothesis consistent with positive samples."""
    hypothesis = None

    for example in training_data:
        attributes = example[:-1]
        label = example[-1]

        # FIND-S ignores negative examples
        if label != positive_label:
            continue

        # The first positive example initialises the hypothesis
        if hypothesis is None:
            hypothesis = attributes.copy()
        else:
            # Generalise any attribute that differs
            for i in range(len(hypothesis)):
                if hypothesis[i] != attributes[i]:
                    hypothesis[i] = "?"

    return hypothesis


# Example training data: EnjoySport
training_data = [
    ["Sunny", "Warm", "Normal", "Strong", "Warm", "Same", "Yes"],
    ["Sunny", "Warm", "High",   "Strong", "Warm", "Same", "Yes"],
    ["Rainy", "Cold", "High",   "Strong", "Warm", "Change", "No"],
    ["Sunny", "Warm", "High",   "Strong", "Cool", "Change", "Yes"]
]

attributes = ["Sky", "AirTemp", "Humidity", "Wind", "Water", "Forecast"]
final_hypothesis = find_s(training_data)

print("Attributes:", attributes)
print("Final hypothesis:", final_hypothesis)
```

7. Sample Input

The input is the training_data list in the code. The last value in each row is the class label.

Sky	AirTemp	Humidity	Wind	Water	Forecast	Label
Sunny	Warm	Normal	Strong	Warm	Same	Yes
Sunny	Warm	High	Strong	Warm	Same	Yes
Rainy	Cold	High	Strong	Warm	Change	No
Sunny	Warm	High	Strong	Cool	Change	Yes

8. Sample Output

```
Attributes: ['Sky', 'AirTemp', 'Humidity', 'Wind', 'Water', 'Forecast']
Final hypothesis: ['Sunny', 'Warm', '?', 'Strong', '?', '?']
```

9. Result

Thus, the FIND-S algorithm was implemented and demonstrated successfully using Python. From the given training data, the most specific hypothesis consistent with all positive examples was found as:

■Sunny, Warm, ?, Strong, ?, ?■

Limitations

- FIND-S ignores negative examples, so it does not verify that the hypothesis rejects all negative samples.
- It assumes the training labels are correct and can be misleading if positive examples contain noise.
- It returns one hypothesis, not every hypothesis consistent with the data.
- The five small examples are for learning the algorithm; they should not be treated as real-world decision rules.